

Rediff Chat Architecture

System Design, Database Models, Event Hooks & Runtime Execution Flows

1. SYSTEM OVERVIEW

Rediff Chat is an enterprise instant messaging application that utilizes **ejabberd** as its underlying message routing cluster engine. The architecture enforces structural decoupling: user identity contexts live cleanly within the standard corporate repository, while real-time packet distribution state maps independently inside the messaging database. Synchronous writes between system boundaries are completely eliminated in favor of an asynchronous **Provisioning Layer**.



2. SYSTEM OWNERSHIP BOUNDARIES

Data access and transactional authority are strictly divided between the core application layer and the communications engine:

REDIFF CORE DOMAIN

System of record for enterprise operations and personnel metadata.

- User Master Identities
- Authentication Proofs & Passwords
- User Profile Configurations
- Employee Context & Metadata
- Global Security Roles / Permissions

EJABBERD DOMAIN

Real-time transient state, message delivery paths, and routing topologies.

- XMPP Specific Accounts (JIDs)
- Volatile Network Presence States
- Rosters & Peer Contacts
- Active WebSocket/TCP Sessions
- Message Distribution Matrix
- Offline Message Spooling Queues
- Multi-User Chat (MUC) Spaces

3. REDIFF DATABASE DESIGN

The core enterprise relational schema tracks authentication and directory mappings. All properties resolve strictly to a single user identity anchor via explicit relational keys.

Table: users [Master Registry]

`id` (BIGINT, PK) | `username` (VARCHAR(100), UNIQUE) | `email` (VARCHAR(255), UNIQUE) | `full_name` (VARCHAR(255)) | `status` (VARCHAR(20)) | `created_at` (TIMESTAMP) | `updated_at` (TIMESTAMP)

Table: user_auth [Credential Core]

`user_id` (BIGINT, FK -> users.id, PK) | `password_hash` (VARCHAR(255)) | `last_login_at` (TIMESTAMP)

Table: user_profile [Personnel Metadata]

`user_id` (BIGINT, FK -> users.id, PK) | `phone` (VARCHAR(50)) | `designation` (VARCHAR(100)) | `avatar_url` (VARCHAR(500))

Rediff Internal Identity Layout



4. EJABBERD DATABASE DESIGN

Optimized for low-latency writes. These tables represent the transactional runtime matrix managed inside Mnesia or an integrated cluster storage tier.

Table: passwd [XMPP Registry – Mnesia Schema: [us, password]]

`jid` (VARCHAR(255), PK) | `password_hash` (VARCHAR(255))

Table: roster [Contact Storage – Mnesia Schema: [usj, us, jid, name, ...]]

owner_jid (VARCHAR(255), PK) | contact_jid (VARCHAR(255), PK) | subscription (VARCHAR(50)) | group_name (VARCHAR(100))

Table: session [Live Telemetry Sockets]

session_id (VARCHAR(255), PK) | jid (VARCHAR(255)) | resource (VARCHAR(100)) | login_time (TIMESTAMP)

Table: route [Internal Node Routing Maps]

jid (VARCHAR(255)) | resource (VARCHAR(100)) | node (VARCHAR(100)) | PK(jid, resource, node)

Table: offline_msg [Deferred Packet Spool]

message_id (BIGINT, PK) | sender_jid (VARCHAR(255)) | recipient_jid (VARCHAR(255)) | message_body (TEXT) | created_at (TIMESTAMP)

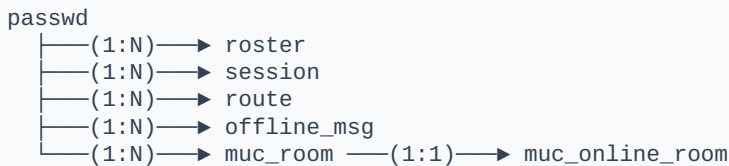
Table: muc_room [Group Registry]

room_jid (VARCHAR(255), PK) | room_name (VARCHAR(255)) | created_by (VARCHAR(255)) | created_at (TIMESTAMP)

Table: muc_online_room [Volatile Group State]

room_jid (VARCHAR(255), PK) | active_users (INT)

ejabberd Engine Relational Layout



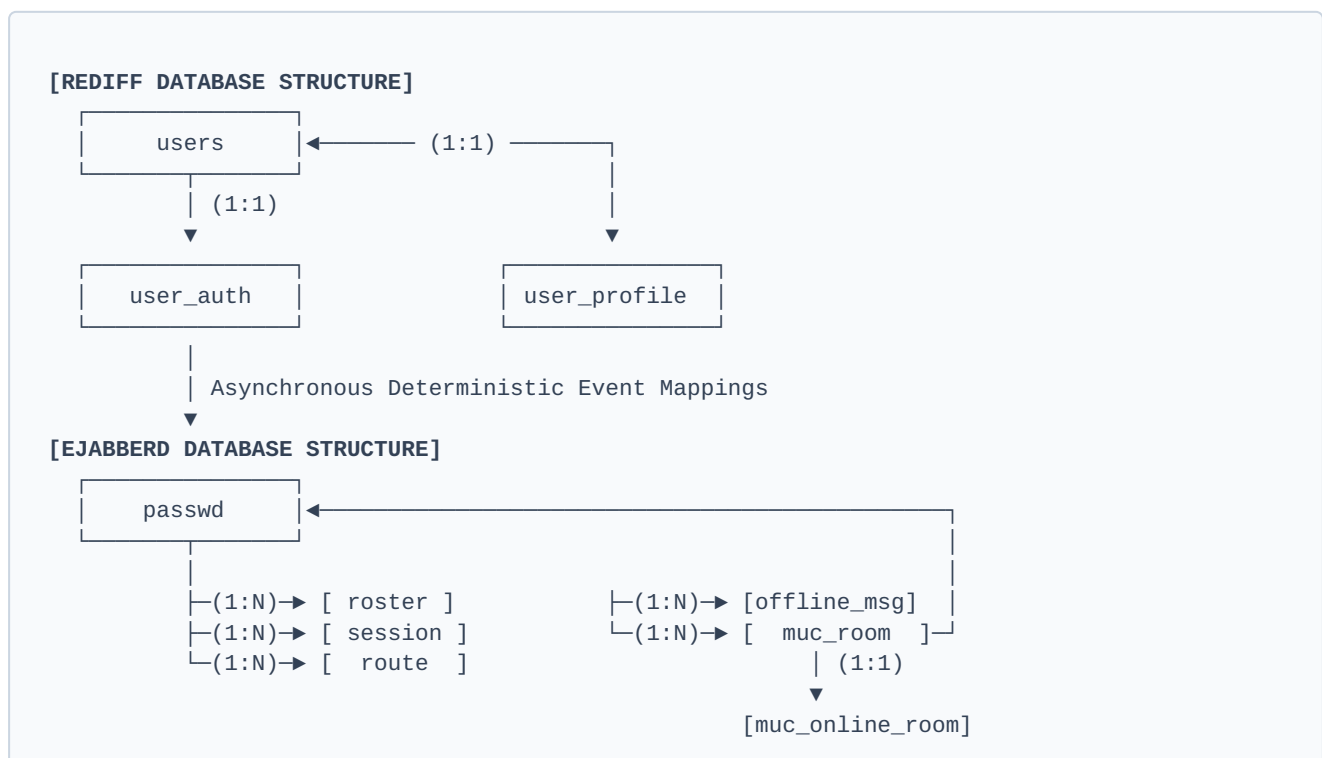
5. CROSS-SYSTEM IDENTITY TRANSLATION RULES

Physical foreign key relationships do not exist across databases. Synchronization resolves via deterministic string transformations applied at the provisioning boundary:

Rediff Core Engine (`users.username`) → String Normalization Rule → **XMPP Target JID**
(`username@chat.rediff.com`)

Explicit Mapping Example: An employee record containing internal identity primary key 101 with standard user handle **alice** resolves downstream within the messaging application container as the target entity destination **alice@chat.rediff.com**.

6. CONSOLIDATED SYSTEM ENTITY TOPOLOGY



7. EVENT & HOOK ARCHITECTURE

Rather than implementing raw blocking loops at the database level, decoupled operations are managed by tracking decoupled transactional events:

SYSTEM EVENT	UPSTREAM PIPELINE ORIGIN	DOWNSTREAM EXECUTION ACTION
UserCreated	INSERT INTO users confirmation	Provision XMPP Account onto destination node clusters
UserDeleted	Organization exit processing routine	Deregister JID, tear down active rosters & sessions
UserUpdated	Profile field transformation execution	Synchronize modified data attributes down into XMPP vCards
MessageSent	Internal ejabberd router intercept hook	Dispatch payloads to secure storage for compliance auditing
RoomCreated	MUC instance activation completion	Register room logs and configuration rules centrally

8. PROCESS FLOW EXECUTIONS

A. User Provisioning Workflow

- Step 1:** Rediff application commits write transaction: `INSERT INTO users (...)`.
- Step 2:** Application layers dispatch a **UserCreated** event payload onto the message broker.
- Step 3:** Provisioning Service workers intercept the event packet from the queue.
- Step 4:** Worker triggers standard secure system command call to ejabberd cluster engine daemon:

```
ejabberdctl register alice chat.rediff.com structural_password_hash
```

B. Runtime Login Validation Workflow

- Step 1:** The client authenticates against Rediff by providing username + password parameters.
- Step 2:** Rediff infrastructure references records inside `user_auth` to validate the identity and issues an authorization token.
- Step 3:** The chat client initiates an active connection towards the ejabberd entry node.
- Step 4: Validation Routing Matrix:**
 - *Option A:* Look up authentication attributes inside local `passwd` system tables.
 - *Option B (Recommended):* Configure ejabberd for **External Authentication**, querying the primary Rediff state engine in real time to guarantee single-source credentials.

C. End-to-End Messaging Pipeline

- Step 1:** Alice pushes payload packet "**Hello Bob**" straight down her active socket to the server.
- Step 2:** The ejabberd cluster engine captures the inbound traffic and queries the active route schema map.
- Step 3:** The system extracts target session properties from the active session data store.
- Step 4: Terminal Delivery Execution:**
 - *If Bob is Online:* The server pushes the raw packet to his cluster-allocated socket node immediately.
 - *If Bob is Offline:* The server logs the message payload as a transient entry inside `offline_msg`.

D. Contact & Group Chat Management

When Alice includes Bob within her active contacts list, records commit directly into the roster table (owner=alice, contact=bob). If group chats are created, configurations write to muc_room, while runtime counters store current traffic metrics inside muc_online_room. **These processes require zero interaction with the Rediff application database.**

9. ARCHITECTURAL PRINCIPLES SUMMARY

Principle 1: Rediff DB is the Absolute Source of Truth

Tracks and defines global Identity, Cryptographic Authentication, and Personnel Profiles.

Principle 2: ejabberd DB operates as an Isolated Messaging Engine

Manages ephemeral Network Presence, Roster Contiguity, Node Sessions, and Multi-User Chat configurations.

By routing interaction boundaries cleanly through the decoupled Event Layer, the configuration eliminates cross-database bottlenecks. This ensures horizontal scalability for both transaction engines under enterprise workloads.